

Syft Accuracy on C/C++ Binary Extraction

Tyler Garriott

Research Question

- Can Syft correctly identify software components inside a compiled binary?
 - Scope:
 - Focus on C/C++ binaries
 - Scanned individual compiled binaries not source trees
 - Compared Syft output against readelf -d and handwritten ground truths
 - Environment:
 - OS: CachyOS Linux (Arch)
 - Architecture x86_64
 - Syft: 1.44.0
 - GCC: 16.1.1

Key Distinctions

- Package Artifacts vs Imported Libraries
 - Package artifacts are SBOM component entries, which means that Syft believes it identified a software package.
 - Imported libraries are lower-level ELF linking evidence. This means that the binary asks the dynamic linker to load shared libraries.
 - An important distinction is that an imported library is not the same as a package artifact.
- In this project Syft was usually reliable at extracting imported libraries but much less consistent at producing package-level artifacts for C/C++ binaries.

Methods

- I built or found C/C++ binaries
- Ran Syft using syft-json
- Ran readelf -d to verify ELF NEEDED imports
- Created ground-truth JSON files
- Compared expected package artifacts and imported libraries
- Examples I tested on:
 - zlib-test, openssl-test, curl-test, fastfetch, zstd, curl, jq, nginx, memcached, openssl, xz, ffmpeg

Syft Extracts ELF Imports Correctly

Binary	Expected Linked Component	Syft package artifacts	Syft Imported Libraries
Zlib-test	zlib	none	libz.so .1, libc.so.6
openssl-test	OpenSSL	none	libcrypto.so .3, libc.so.6
curl-test	curl/libcurl	none	libcurl.so .4, libc.so.6
sqlite-test	SQLite	none	libsqlite3.so .0, libc.so.6
ffmpeg	FFmpeg	none	10 ELF imported libraries

Syft matched the ELF NEEDED imports from readelf -d, but did not turn these imports into package-level SBOM artifacts.

Syft Finds Some Binaries Through Specific Classifiers

Binary	Package artifact found for expected filename?	Version exact?
curl	Yes	No
jq	Yes	Yes
nginx	Yes	Yes
memcached	Yes	Yes
openssl	Yes	No
xz	Yes	Yes
ffmpeg	No	N/A

Syft appears to have specific binary classifiers, but coverage is uneven.

Filename Sensitivity

Component	Expected Filename Result	Renamed File Result	Imported Libraries
jq	jq found	jq-linux-amd64: none	None for both
memcached	memcached found	memcached-built: none	Same
openssl	openssl found	openssl-built: none	Same
xz	xz found	xz-built: none	Same

**Same bytes.
Same imports.
Different package
results.**

Renaming the file caused Syft to lose the package artifact, while ELF imported-library extraction stayed the same.

Controlled OpenSSL Rename Test

Test Path	Syft Package Artifact
random-parent/openssl	openssl found
openssl-parent/openssl-built	none
stripped openssl	openssl found
stripped openssl-built	none

The basename openssl mattered more than the parent path or symbol stripping.

FFmpeg Case

- Why FFmpeg?
 - FFmpeg is a real-world C/C++ binary with many build features.
- Results:
 - ffmpeg -version lists many enabled libraries
 - readelf -d showed 10 direct ELF imports
 - Syft extracted all 10 correctly
 - Syft reported zero package artifacts for ffmpeg
- Takeaway:
 - Build configuration, ELF imports, and SBOM package artifacts answer different questions.

Limitations

- I only tested with Syft, mainly 1.44.0
- Focused exclusively on Linux ELF C/C++ binaries
- Did not test every binary format or every SBOM tool
- Ground truth was manually created
- Imported libraries only show dynamic dependencies
- Static linking and embedded code is harder to detect from ELF imports alone

Conclusion

- Syft can identify some known C/C++ binaries as package artifacts, but detection appears to depend on specific classifiers, expected filenames, and version strings.
- Syft is useful for ELF imported-library evidence, but binary-only package artifact detection is tough and should be interpreted carefully.
- For C/C++ binaries, imported-library extraction should not be treated as complete SBOM component detection.